

# IA para Desenvolvedores

Como Usar Inteligência Artificial para  
Programar Melhor — sem Cair nos Erros  
Comuns

Ilvan Joaquim

---

# IA para Desenvolvedores

*Como Usar Inteligência Artificial para Programar Melhor -- sem  
Cair nos Erros Comuns*  
Autor: Ivanildo Aquino  
Edição: 1ª -- 2026

# **O Erro de 90% ao Usar IA para Programar**

---

# O erro que 90% das pessoas cometem usando IA para programar

---

**Nível:** Conceitos / Engenharia

**Tempo estimado:** 20 minutos

**Público-alvo:** Desenvolvedores iniciantes e intermediários que utilizam assistentes de IA no dia a dia

---

## Pré-requisitos

- Experiência básica com programação em qualquer linguagem
- Familiaridade com uso de assistentes de IA (GitHub Copilot, Claude Code, ChatGPT, etc.)
- Noções fundamentais de Git e versionamento

## Objetivos de aprendizagem

Ao final desta aula, o aluno será capaz de:

1. **Identificar** os 5 erros mais comuns ao usar IA para programar
2. **Diferenciar** uso produtivo de uso prejudicial de assistentes de IA
3. **Aplicar** técnicas de prompt estruturado para obter respostas precisas

4. **\*\*Estabelecer\*\*** um fluxo de validação e revisão para código gerado por IA
5. **\*\*Configurar\*\*** arquivos de instrução do projeto (AGENTS.md / CLAUDE.md) para melhorar a consistência do agente

## Competências desenvolvidas

### Hard skills:

• Prompt engineering aplicado à programação

- Code review de código gerado por IA
- Configuração de assistentes de IA no projeto
- Automação de testes como ferramenta de verificação

### Soft skills:

• Pensamento crítico e validação de fontes

- Comunicação clara e estruturada
- Responsabilidade profissional sobre o código produzido
- Iteração e refinamento contínuo

---

## 1. Introdução: por que 90% cometem esse erro

Era uma terça-feira comum. O desenvolvedor precisava de uma função simples para validar emails. Pediu ao ChatGPT, copiou o código, fez deploy. Na sexta, o banco de dados estava cheio de registros com emails como "usuario@". Aquela função -- que parecia perfeita -- só verificava se existia um "@" na string. A IA acertou a sintaxe. Errou a lógica. O desenvolvedor não revisou.

Produção quebrou. Esse cenário se repete milhares de vezes todos os dias. Assistentes de

IA geram código em segundos -- mas essa velocidade tem um custo

| Dado                                                                           | Fonte                                  |
|--------------------------------------------------------------------------------|----------------------------------------|
| <b>**63%**</b> dos desenvolvedores encontraram erros inesperados ao usar IA    | Stack Overflow Survey                  |
| <b>**68%**</b> têm dificuldade em integrar IA efetivamente nos workflows       | Stack Overflow Survey                  |
| Sem regras configuradas, código gerado por IA tem <b>**~40%</b> de erro        | Claude Code Pro Pack                   |
| Com 12 regras, o erro cai para <b>**~3%**</b> -- melhoria de <b>**~13,3x**</b> | Claude Code Pro Pack Research (DEV.to) |

O problema não é a IA. O problema é **como usamos a IA**. A maioria dos desenvolvedores repete os mesmos 5 erros -- e 90% sequer

**Dica:** Esta aula compila fontes oficiais (GitHub, Anthropic, OpenCode), pesquisa acadêmica (arXiv 2512.05239) e benchmarks da indústria para mapear os erros e, mais importante, mostrar como corrigi-los.

**Pare e pense:** quantas vezes você copiou código de IA sem ler cada linha? Se a resposta for "algumas vezes", esta aula é para você.

## 2. Erro 1: Confiar cegamente na saída da IA

### Definição

Aceitar o código gerado pela IA como correto sem qualquer questionamento, validação ou revisão.

### Por que acontece

A IA gera respostas com **alta fluência e aparência de confiança**. O código parece correto, compila e muitas vezes até passa em testes simples -- mas pode conter bugs sutis de lógica, segurança ou performance.

"A IA erra com confiança, não com hesitação." -- TechTudo

O problema é psicológico: nosso cérebro associa fluência a competência. Quando a IA escreve um parágrafo ou função que "soa bem", relaxamos a guarda. Só que a IA não sabe o que está fazendo -- ela está apenas completando padrões estatísticos.

| Tipo de Bug         | O que significa                                          | Exemplo                                          |
|---------------------|----------------------------------------------------------|--------------------------------------------------|
| **Lógica**          | Código sintaticamente correto, mas semanticamente errado | Valida email só com <code>`includes('@')`</code> |
| **Segurança**       | Vulnerabilidades introduzidas                            | SQL injection, senha em MD5                      |
| **Performance**     | Código ineficiente                                       | Loop aninhado desnecessário, N+1 queries         |
| **Compatibilidade** | Dependências incorretas ou desatualizadas                | Import de biblioteca que não existe mais         |

## Consequência

Bugs em produção, vulnerabilidades de segurança e dívida técnica acumulada. Como alerta a documentação oficial do GitHub Copilot:

*"Remember that you are in charge, and Copilot is a powerful tool at your service."*

Traduzindo: o responsável é você. A ferramenta é só a ferramenta.

## Exemplo concreto

```
[javascript]

// [prohibited] Código gerado por IA -- parece certo, mas está er...
function calculateDiscount(price, coupon) {
  if (coupon === 'SAVE10') {
    return price * 0.9;
  }
  return price; // [X] Esqueceu de validar se price é número
}

calculateDiscount('cem reais', 'SAVE10'); // NaN em produção
```

## Como corrigir

Passo a passo para todo código gerado por IA:

- **\*\*Leia cada linha antes de implementar\*\*** -- se não entendeu, não use
- **\*\*Teste com casos extremos\*\*** -- string vazia, null, negativo, tipos inesperados
- **\*\*Valide dependências sugeridas\*\*** -- a IA pode inventar bibliotecas que não existem
- **\*\*Use type checking\*\*** (TypeScript, mypy, etc.) para pegar erros de tipo

**Dica:** Trate a saída da IA como um **rascunho inicial**, não como produto acabado. A diferença entre um profissional e um amador é que o profissional verifica antes de entregar.

## 3. Erro 2: Prompts vagos sem contexto

### Definição

Fazer pedidos genéricos e esperar respostas precisas e úteis.

### Por que acontece

A IA opera com base em probabilidades: quanto menos contexto, mais genérica a resposta. É como perguntar "Me recomenda um filme?" para um amigo -- você vai receber uma lista genérica. Agora pergunte "Me recomenda um filme de suspense coreano com menos de 2 horas" e a resposta muda completamente.

O mesmo vale para código.  
*Pedido fraco, resposta fraca em escala industrial.* --  
TechTudo

## Consequência

Respostas genéricas que não resolvem o problema real. O desenvolvedor perde tempo iterando sobre sugestões irrelevantes, se frustra com a ferramenta e culpa a IA -- quando o problema era o prompt.

### Exemplo concreto

```
<table>
```

```
<tr>
```

```
<td width="50%">
```

Manobre esse código?  
A IA não sabe

- Qual linguagem?
- Qual critério de "melhor"?
- Performance? Legibilidade? Segurança?
- Qual o contexto do projeto?

```
</td>
```

```
<td width="50%">
```

Prompt ruim  
Relatore a função handleSubmit no arquivo src/forms.ts para usar async/await com try-catch, mantendo o mesmo comportamento e seguindo o padrão de error handling do resto do projeto."  
A IA sabe

- Arquivo e função exatos

- O que fazer (refatorar)
- Como fazer (async/await + try-catch)
- Restrições (manter comportamento, seguir padrão existente)

</td>

</tr>

</table>

## Como corrigir

| Elemento             | Pergunta guia             | Exemplo                                                              |
|----------------------|---------------------------|----------------------------------------------------------------------|
| **Contexto**         | Qual é o cenário?         | "No arquivo `login.ts`,<br>função<br>`authenticateUser`..."          |
| **Intenção**         | O que você quer alcançar? | "...precisa validar token<br>JWT antes de consultar o<br>banco"      |
| **Formato esperado** | Como deve ser a resposta? | "...retorne `Result<T, E>`,<br>sem `throw`, com testes<br>em Vitest" |

**Dica:** Antes de escrever um prompt, pergunte-se: "Se eu desse esta instrução para um colega desenvolvedor, ele saberia exatamente o que fazer?" Se a resposta for não, ~~adicione mais contexto.~~

## 4. Erro 3: Pular code review e testes

### Definição

Ignorar as etapas de revisão de código e testes automatizados para código gerado por IA, tratando-o como isento de erros.

### Por que acontece

A velocidade da IA cria a ilusão de que o código já passou por um "controle de qualidade implícito". O desenvolvedor assume que, se a

IA gerou, está correto.  
*Se você copia e cola sem ler, o erro deixa de ser da ferramenta. Passa a ser seu.* -- TechTudo

Essa falsa sensação de segurança é traiçoeira. O código gerado por IA **não foi revisado por ninguém**. Ele é o equivalente a um primeiro rascunho escrito por alguém que nunca usou seu sistema.

A documentação do Claude Code é categórica:  
*Give Claude a check it can run, tests, a build, a screenshot to compare. It's the difference between a session you watch and one you walk away from.*

Sem uma verificação executável, "parece pronto" é o único sinal disponível. Você se torna o loop de verificação -- cada erro espera **você** perceber.

## Consequência

Dívida técnica acumulada, bugs não detectados, vulnerabilidades de segurança e código de difícil manutenção. O commit vai para o repositório com **seu nome** -- a IA não assume responsabilidade.

## Como corrigir

- **\*\*Code review obrigatório\*\*** -- revise código gerado por IA como revisaria de um colega. Pergunte: "Eu aceitaria isso num PR?"
- **\*\*Testes automatizados como verificação\*\*** -- antes de aceitar o código, peça para a IA gerar os testes também
- **\*\*CI pipeline\*\*** -- todo código, inclusive o gerado por IA, deve passar pelos mesmos checks automatizados

**Atenção:** Pular code review em código gerado por IA não economiza tempo -- ele **terceiriza o risco** para você. O bug vai aparecer, a pergunta vai ser "quem autorizou isso?", e a resposta será **seu nome no commit**.

## 5. Erro 4: Ignorar configuração do projeto

# (AGENTS.md / CLAUDE.md)

## Definição

Não configurar arquivos de instrução persistente para os agentes de IA, deixando-os operar sem contexto do projeto.

### Por que acontece

Arquivos como `AGENTS.md` (OpenCode) e `CLAUDE.md` (Claude Code) funcionam como a memória de longo prazo do agente. Eles contêm regras, padrões e convenções do projeto. Sem eles, a IA opera com conhecimento genérico. Ela não sabe se o projeto usa React 18 ou Vue 3. Não sabe se prefere `const` ou `function`. Não sabe se os testes são com Vitest ou Jest. Ela chuta.

*5 minutos de configuração economizam horas de retrabalho."*  
-- OpenCode Community

## Consequência

Comportamento inconsistente do agente -- o código gerado muda de estilo a cada interação, ignora padrões do projeto e força retrabalho manual.

| Configuração             | Taxa de erro | Melhoria |
|--------------------------|--------------|----------|
| -----                    | :-----:      | :-----:  |
| Sem regras               | ~40%         | --       |
| Com 4 regras básicas     | ~11%         | ~3,6x    |
| Com 12 regras (pro pack) | ~3%          | ~13,3x   |

Fonte: Claude Code Pro Pack Research (DEV.to)

### Como corrigir

Crie um arquivo de instrução na raiz do projeto. A estrutura mínima inclui:

**Exemplo de ``AGENTS.md`` para um projeto front-end:**

[markdown]

# Agente: Front-end

## Stack

React 18 + TypeScript + Tailwind CSS

## Regras

- Use arrow functions para componentes
- Prefira `const` sobre `let`
- Testes com Vitest, não Jest
- Erros devem usar `Result<T, E>` (never throw)
- Nomes de arquivo em kebab-case
- Componentes em `src/components/`, páginas em `src/pages/`

**Dica:** Invista 5 minutos agora para criar o AGENTS.md. É o investimento com maior retorno por minuto no uso de IA para programar. A cada novo projeto, comece por ele.

**Pare e pense:** seu projeto atual tem um arquivo de instruções para a IA? Se não, esse é o erro número 1 que você está cometendo sem perceber.

## 6. Erro 5: Tratar IA como resposta final

### Definição

Usar a primeira resposta da IA como solução definitiva, sem refinamento ou iteração.

### Por que acontece

A IA entrega respostas completas e aparentemente prontas. O desenvolvedor assume que a primeira tentativa é a melhor e encerra o ciclo ali. É o mesmo impulso de mandar um email sem reler -- a gratificação imediata de "pronto" supera a disciplina de refinar.

*"A primeira resposta raramente é a melhor. Ela é o ponto de partida, não o produto final." -- TechTudo*

## Consequência

Resultados superficiais. O código funciona no caminho feliz, mas quebra nos casos extremos. O desenvolvedor perde a oportunidade de refinar, corrigir e adaptar a solução ao contexto real. Pior: nunca sabe o que perdeu.

### Exemplo concreto

Comparação entre primeira resposta vs. versão refinada de um

```
[javascript]

// [prohibited] Primeira resposta (rascunho)
function validateEmail(email) {
  return email.includes('@');
  // [X] Aceita "@", "a@", "a@b"
}

// [OK] Versão refinada com feedback
function validateEmail(email) {
  const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
  return emailRegex.test(email);
  // [OK] Rejeita "@", "a@", "a@b", aceita "user@domain.com"
}
```

## Como corrigir

Trate IA como **processo iterativo**, não como resposta final:

- Obtenha uma primeira versão -- o rascunho inicial
- **\*\*Revise e identifique pontos de melhoria\*\*** -- o que está faltando?

- **\*\*Refine com feedback direcionado\*\*** -- seja específico sobre o que mudar
- **\*\*Repita até atender aos critérios de qualidade\*\*** -- quando passar nos testes e na revisão

**Dica:** Em vez de reescrever o prompt do zero, **itere através de feedback**. Diga à IA especificamente o que ajustar: "Mude a nomenclatura para camelCase", "Extraia essa lógica para um hook separado" ou "Adicione tratamento para o caso de lista vazia." A pesquisa SFEIR Institute confirma: desenvolvedores que iteram com feedback estruturado reduzem em **~35% as iterações**

**necessárias** em comparação com quem reescreve prompts do zero.

## 7. Conclusão: como usar IA corretamente

Usar IA para programar não é sobre aceitar código -- é sobre **colaboração inteligente**. A IA é uma ferramenta poderosa, mas sem direção, validação e contexto, ela produz resultados medíocres.

### Os 5 mandamentos do uso correto de IA

| # | Mandamento                                        | Por quê                                   |
|---|---------------------------------------------------|-------------------------------------------|
| 1 | <b>**Valide**</b> toda saída da IA                | IA erra com confiança, não com hesitação  |
| 2 | <b>**Estruture**</b> prompts com contexto         | Pedido fraco -> resposta fraca            |
| 3 | <b>**Revise e teste**</b> como qualquer código    | Seu nome está no commit                   |
| 4 | <b>**Configure**</b> o projeto para a IA          | 5 minutos economizam horas                |
| 5 | <b>**Itere**</b> , não aceite a primeira resposta | Refinamento separa o mediano do excelente |

**Dica:** Desenvolvedores que aplicam essas práticas reduzem em **~35% as iterações necessárias** (SFEIR Institute) e produzem código com **~97% de acerto** (Claude Code Pro Pack).

## Diagrama conceitual: Fluxo ideal de uso de IA

[text]

```
[Prompt Estruturado] -> [IA gera rascunho] -> [Code Review] -> [T...  
  ^  
  |  
  |_____ Iteração (feedback) _____|
```

## Analogia principal

Usar IA para programar é como ter um **estagiário brilhante, mas inexperiente**. Ele trabalha rápido, escreve bem, mas precisa de supervisão, contexto e revisão constantes. Confiar cegamente nele é negligência; ignorá-lo é desperdício. O profissional sábio sabe exatamente quando delegar, quando revisar e como orientar.

## Recursos didáticos sugeridos

[javascript]

```
// Prompt vago: "Valide esse email"  
function validateEmail(email) {  
  return email.includes('@'); // [X] Superficial, não valida domín...  
}  
  
// Prompt estruturado: "Valide email com regex básica de formato,..."  
function validateEmail(email) {  
  const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;  
  return emailRegex.test(email); // [OK] Validação mais robusta  
}
```

**Sugestão de diagrama:** Mapa mental dos 5 erros com causas e correções lado a lado, em formato de tabela visual.

**Mini-exercício mental para reflexão:** Pense no último código que você copiou de uma IA. Você revisou linha por linha? Escreveu testes? Se respondeu "não" para qualquer pergunta, identifique qual dos 5 erros você cometeu.

## Exercício prático

## Título: Diagnosticando um prompt ruim

Duração: 5 minutos

Instruções: Leia o prompt abaixo:

> "Faz aí uma função de login pra mim"

1. Liste 3 problemas com este prompt (erro 2 -- prompt vago)
  - Reescreva o prompt seguindo a estrutura **\*\*Contexto + Intenção + Formato Esperado\*\***
  - Se possível, execute o prompt reescrito em um assistente de IA e compare a qualidade da resposta

## Critérios de sucesso:

- Identificou a falta de: linguagem/framework, definição de sucesso, tratamento de erros
- Novo prompt inclui pelo menos: stack tecnológica, regras de negócio, formato da resposta esperada

| Problema                   | Explicação                                               |
|----------------------------|----------------------------------------------------------|
| Falta linguagem/framework  | "Faz aí" não diz se é Node, Python, PHP, etc.            |
| Falta definição de sucesso | O que é "login"? JWT? Sessão? OAuth?                     |
| Falta tratamento de erros  | E se o usuário não existe? Senha errada? Taxa de limite? |

## Prompt reescrito (exemplo):

*Crie uma função de login em Node.js com Express. O usuário envia email e senha no corpo da requisição. Valide o email com regex, compare a senha com bcrypt, e retorne um token JWT com expiração de 24h. Se o email não existir ou a senha estiver errada, retorne 401 com mensagem clara. Use `async/await` com `try-catch`."*

## Desafio final

**Título:** Auditoria de código gerado por IA

**Cenário:** Você recebeu um Pull Request com código 100% gerado por IA. O desenvolvedor apenas copiou e colou sem revisar.

**Tarefa:** Analise o trecho abaixo, identifique todos os erros e

```
[python]

import os
import hashlib

def hash_password(password):
    # Gerar hash simples
    return hashlib.md5(password.encode()).hexdigest()

def save_user(username, password):
    hashed = hash_password(password)
    query = f"INSERT INTO users (username, password) VALUES ('{us...
    os.system(f"mysql -e \"{query}\"")
```

**O que procurar:**

- 1. **Erro 1:** confiar cegamente -- código tem vulnerabilidades graves
- 2. **Erro 3:** pular code review -- falta de testes e validação
- 3. **Erro 5:** tratar como resposta final -- código inseguro em produção

| Erro identificado          | Por que é grave                                              | Correção                                                          |
|----------------------------|--------------------------------------------------------------|-------------------------------------------------------------------|
| MD5 para senhas            | MD5 é instantaneamente quebrável com ataques de dicionário   | Use `bcrypt` ou `argon2`                                          |
| SQL injection              | String formatada diretamente permite injeção de comandos SQL | Use prepared statements / ORM                                     |
| `os.system`                | Expõe o shell a comandos maliciosos; desnecessário           | Use biblioteca MySQL com parâmetros<br>(`mysql-connector-python`) |
| Falta validação de entrada | `username` pode conter caracteres maliciosos                 | Valide e sanitize entradas                                        |

```
[python]

import os
import bcrypt
import mysql.connector

def hash_password(password):
    return bcrypt.hashpw(password.encode(), bcrypt.gensalt())

def save_user(username, password):
    if not username or not password:
        raise ValueError("Campos obrigatórios")

    hashed = hash_password(password)
    conn = mysql.connector.connect(
        host=os.getenv("DB_HOST", "localhost"),
        user=os.getenv("DB_USER", "app"),
        password=os.getenv("DB_PASSWORD"),
        database=os.getenv("DB_NAME", "appdb")
    )
    cursor = conn.cursor()
    cursor.execute(
        "INSERT INTO users (username, password) VALUES (%s, %s)",
        (username, hashed)
    )
    conn.commit()
    cursor.close()
    conn.close()
```

## Leituras complementares

- GitHub Copilot Best Practices --  
<https://docs.github.com/en/copilot/get-started/best-practices>

- Claude Code Best Practices --  
<https://code.claude.com/docs/en/best-practices>
- OpenCode Official Docs -- <https://opencode.ai/>
- "A Survey of Bugs in AI-Generated Code" (arXiv 2512.05239) --  
<https://arxiv.org/abs/2512.05239>
- 5 erros ao usar IA que sabotam suas respostas (TechTudo) --  
<https://www.techtudo.com.br/listas/2026/04/5-erros-ao-usar-ia-que-sabotam-suas-respostas-e-como-evita-los-edsoftwares.html>
- CLAUDE.md Rules: How to Cut AI Coding Mistakes from ~40% to ~3% (DEV.to) --  
<https://dev.to/rams901/claudemd-rules-how-to-cut-ai-coding-mistakes-from-40-to-3-in-2026-2j70>
- 10 Most Common Mistakes Using AI Coding Tools (Ryz Labs) --  
<https://www.ryzlabs.com/>

# **Agentes de IA na Prática**

---

# Módulo 18 -- Agentes de IA: Criação de Agentes Especializados

---

**Construindo seu exército de agentes.**

## 1. Por que agentes especializados?

Um único agente genérico (ex: "você é um desenvolvedor full stack")

produz resultados **medianos em todas as áreas**.  
Um ecossistema de agentes especializados produz **resultados excelentes em cada área**.

### O problema do agente genérico

[text]

Agente Genérico:

Conhecimento: "sabe de tudo um pouco"

- |— Frontend: [star][star]\*\*\* (sabe React, mas não Next.js App...
- |— Backend: [star][star][star]\*\* (sabe criar API, mas não DD...
- |— Segurança: [star]\*\*\*\* (esquece CSRF, rate limiting)
- |— Banco: [star][star]\*\*\* (faz N+1 sem perceber)
- |— DevOps: [star]\*\*\*\* (Dockerfile sem multi-stage)

Resultado: código "funciona", mas cheio de dívida técnica

### A solução dos agentes especializados

[text]

Agente Frontend:

Conhecimento: Next.js 14, RSC, Tailwind, shadcn/ui

- |— Performance: [star][star][star][star][star] (Lazy loading, ...)
- |— Acessibilidade: [star][star][star][star][star] (ARIA, WCAG ...)
- |— SEO: [star][star][star][star][star] (Metadata API, OG tags, ...)
- |— TypeScript: [star][star][star][star][star] (strict mode, ge...

Resultado: código de produção, pronto para review

## 2. Anatomia de um agente

[text]

agente/

- |— README.md # Identidade: objetivo, responsabilidades...
- |— workflow.md # Processo: fluxo de trabalho passo a pas...
- |— checklist.md # Qualidade: o que validar antes de entre...
- |— prompts/ # Instruções: templates de prompt para ta...
- | |— prompt-tarefa-1.md
- | |— prompt-tarefa-2.md

## Componentes de um agente eficaz

[text]

#### IDENTIDADE

Quem é este agente?  
O que ele sabe fazer?  
O que ele NÃO faz?

#### CONHECIMENTO

Stack tecnológica  
Padrões e boas práticas  
Referências externas

#### PROCESSO

Fluxo de trabalho  
Entrada -> Transformação -> Saída

#### QUALIDADE

Checklist de validação  
Critérios de aceite  
Anti-padrões a evitar

#### COMUNICAÇÃO

Formato de entrada (o que recebe)  
Formato de saída (o que entrega)  
Como reportar problemas

## 3. Os 17 agentes da formação

### Agentes de Produto

| Agente                | Função                              | Stack/Conhecimento                           |
|-----------------------|-------------------------------------|----------------------------------------------|
| **Product Discovery** | Transformar problemas em requisitos | User Stories, RICE, MoSCoW, BDD              |
| **UX Research**       | Validar hipóteses com usuários      | Entrevistas, testes de usabilidade, personas |

## Agentes de Design

| Agente          | Função                          | Stack/Conhecimento                               |
|-----------------|---------------------------------|--------------------------------------------------|
| **UX Designer** | Projetar experiência do usuário | User flows, wireframes, acessibilidade (WCAG)    |
| **UI Designer** | Projetar interface visual       | Design System, tokens, dark mode, responsividade |

## Agentes de Arquitetura

| Agente                   | Função                 | Stack/Conhecimento                        |
|--------------------------|------------------------|-------------------------------------------|
| **Enterprise Architect** | Decisões arquiteturais | Clean Arch, DDD, ADRs, C4 Model           |
| **Database Architect**   | Modelagem de dados     | PostgreSQL, índices, partições, migrações |

## Agentes de Desenvolvimento

| Agente              | Função                   | Stack/Conhecimento                                   |
|---------------------|--------------------------|------------------------------------------------------|
| **Backend Expert**  | APIs e regras de domínio | NestJS, Prisma, REST, GraphQL, Zod                   |
| **Frontend Expert** | Interfaces e componentes | Next.js 14, RSC, Tailwind, shadcn/ui, TanStack Query |
| **Prisma Expert**   | Schema e queries         | Prisma ORM, migrations,                              |

N+1 prevention, soft delete

## Agentes de Infraestrutura

| Agente              | Função                | Stack/Conhecimento                                |
|---------------------|-----------------------|---------------------------------------------------|
| **DevOps Expert**   | Docker, CI/CD, deploy | Docker multi-stage, GitHub Actions, health checks |
| **Security Expert** | Proteção do sistema   |                                                   |
|                     |                       | OWASP Top 10, JWT, bcrypt, rate limiting, CSP     |

## Agentes de Qualidade

| Agente                 | Função               | Stack/Conhecimento                                      |
|------------------------|----------------------|---------------------------------------------------------|
| **QA Expert**          | Testes automatizados | Jest, Playwright, Testing Library, cobertura >80%       |
| **Performance Expert** | Otimização           |                                                         |
|                        |                      | Core Web Vitals, caching, bundle analysis, load testing |

## Agentes de Governança

| Agente            | Função                | Stack/Conhecimento                                                |
|-------------------|-----------------------|-------------------------------------------------------------------|
| **Auditor**       | 16 tipos de auditoria | Score (0-10), riscos (Blocker a Minor), planos de ação            |
| **Documentation** | Documentação técnica  |                                                                   |
|                   |                       | ADRs, README, Swagger, CHANGELOG,                                 |
| **Refactoring**   | Refatoração guiada    | CONTRIBUTING patterns, TypeScript strict, preservar comportamento |

## 4. Como criar um novo agente

## Passo a passo

[text]

1. Definir o DOMÍNIO do agente
  - > Qual área ele cobre? (ex: "segurança de aplicações web")
2. Definir o CONHECIMENTO BASE
  - > Quais tecnologias, padrões e boas práticas ele domina?
3. Definir RESPONSABILIDADES
  - > O que ele faz? O que ele NÃO faz? (limites são importantes)
4. Definir o PROCESSO
  - > Qual o fluxo de trabalho? O que recebe na entrada? O que en...
5. Criar CHECKLIST DE QUALIDADE
  - > O que validar antes de considerar o trabalho concluído?
6. Criar TEMPLATES DE PROMPT
  - > Prompts reutilizáveis para as tarefas mais comuns
7. TESTAR com um caso real
  - > Executar o agente, revisar o output, ajustar

## Exemplo: Criando o Security Expert Agent

**Domínio:** Segurança de aplicações web

**Conhecimento base:**

1. OWASP Top 10 (2021)
2. JWT, OAuth2, MFA
3. bcrypt, Helmet, CORS, CSP
4. NestJS Guards, CASL (autorização)

**Responsabilidades:**

• Implementar autenticação e autorização

- Prevenir SQL injection, XSS, CSRF
- Configurar rate limiting
- Gerenciar segredos

### Limites:

- Não define arquitetura geral
- Não implementa lógica de negócio
- Não gerencia infraestrutura

[text]

- [ ] Senhas com hash bcrypt/argon2
- [ ] JWT com expiração curta + refresh token
- [ ] Rate limiting no login
- [ ] CSP header configurado
- [ ] Helmet.js ativado
- [ ] Input validation em todos os endpoints
- [ ] SQL injection prevenido (ORM)

## 5. Como combinar agentes em pipeline

O verdadeiro poder está em **compor** agentes em sequência.

### Pipeline de features



## Pipeline de auditoria

[text]

Feature implementada

|

v

Security Auditor —-> Se Bloquer/Critical -> Backend Expert (corr...

|

|

v

v

Architecture Auditor —-> Se problema -> Enterprise Architect (re...

|

v

Performance Auditor —-> Se lento -> Performance Expert (otimiza)

|

v

Code Quality Auditor —-> Se abaixo do padrão -> Refactoring Agent

|

v

Relatório consolidado com score geral

## Pipeline de onboarding

[text]

```
PO descreve problema em linguagem natural
  |
  v
Product Discovery Agent -> User Stories + Acceptance Criteria
  |
  v
UX Researcher Agent -> Valida com usuários -> Personas + Jornada
  |
  v
UX Designer Agent -> Wireframes + User Flows
  |
  v
UI Designer Agent -> Mockups com Design System
  |
  v
Enterprise Architect -> Arquitetura + ADRs
  |
  v
Backend + Frontend + Prisma + Security -> Implementação
  |
  v
QA Agent -> Testes
  |
  v
DevOps Agent -> Deploy
  |
  v
Auditor Agent -> Score final
```

## 6. Integração com OpenCode

### Configuração de agentes no opencode.json

```
[json]

{
  "agents": {
    "frontend-expert": {
      "prompt": ".opencode/agents/frontend-expert.md",
      "permissions": {
        "bash": true,
        "read": true,
        "edit": true,
        "glob": true,
        "grep": true
      }
    },
    "auditor": {
      "prompt": ".opencode/agents/auditor.md",
      "permissions": {
        "bash": true,
        "read": true,
        "edit": false,
        "glob": true,
        "grep": true
      }
    }
  }
}
```

## Como invocar um agente

```
[text]

@frontend-expert Crie um componente de tabela com:
- Server Component
- Suporte a sort e filter
- Paginação
- Loading state com Suspense
```

## Como fazer um agente revisar outro

[text]

@auditor Revise a segurança deste endpoint.

[endpoint code]

## 7. Boas práticas na criação de agentes

### Faça

- **\*\*Seja específico\*\*** -- "Crie componente com Server Component" não "Faça um componente bonito"
- **\*\*Defina limites\*\*** -- "Este agente NÃO implementa regras de domínio"
- **\*\*Forneça exemplos\*\*** -- "Siga este padrão: [exemplo]"
- **\*\*Crie checklists\*\*** -- "Antes de entregar, verifique: [itens]"
- **\*\*Itere\*\*** -- Ajuste os prompts baseado nos resultados

### Não faça

- **\*\*Não misture domínios\*\*** -- Um agente de backend não deve ter responsabilidades de frontend
- **\*\*Não seja vago\*\*** -- "Seja criativo" não é uma instrução útil
- **\*\*Não ignore limites\*\*** -- Se o agente não tem conhecimento, ele vai alucinar
- **\*\*Não pule a revisão\*\*** -- Sempre revise o output, especialmente no início

### Padrão de prompt eficaz

[text]

Ruim:

"Crie uma API de usuários."

Bom:

"Crie uma API REST de usuários com NestJS seguindo Clean Architec...

Requisitos:

- POST /users (criar)
- GET /users (listar com paginação)
- GET /users/:id (detalhe)
- PUT /users/:id (atualizar)
- DELETE /users/:id (soft delete)

Validações:

- Email: formato válido, único
- Nome: 3-100 caracteres
- Senha: mínimo 8 caracteres, 1 número, 1 maiúscula

Regras:

- Usar Prisma para persistência
- Zod para validação
- Swagger para documentação
- Tratamento de erros com NestJS exception filters"

## 8. O futuro: agentes que criam agentes

O próximo passo natural: um **Meta-Agent** que cria agentes especializados sob demanda.

### Como funcionaria

[text]

# Automação do Ciclo de Desenvolvimento com IA

# Módulo 20 -- Automação

**\*\*Duração estimada:\*\*** 5 dias (~40h)

**\*\*Público-alvo:\*\*** Desenvolvedores brasileiros em transição para c...

**\*\*Pré-requisitos:\*\*** Conhecimento básico de Git, terminal Linux, c...

---

## 1. O que é Automação

Automação é a substituição de processos manuais repetitivos por s...

### Por que automatizar?

| Motivo              | Impacto                                                   |
|---------------------|-----------------------------------------------------------|
| Reduzir erro humano | O maior causador de incidentes em produçã...              |
| Velocidade          | Máquinas executam em segundos o que levaria horas ...     |
| Reprodutibilidade   | O mesmo pipeline executa exatamente da mesm...            |
| Auditoria           | Logs e artefatos gerados automaticamente servem com...    |
| Escala              | O que funciona para 1 deploy falha para 100 deploys ma... |

### Onde automatizar

- Build e compilação
- Testes (unitários, integração, e2e)
- Linting e formatação de código
- Verificação de segurança
- Deploy em ambientes
- Migrações de banco de dados

- Geração de changelog e release

- Monitoramento e alertas

- Criação e destruição de ambientes



### ### Custo vs Benefício



A regra prática: **\*\*automatize tudo que for executado mais de 2 ve...**

Custo de automatizar = (tempo para criar + tempo para manter) × custo-hora

Benefício = (tempo economizado por execução × frequência ×

```
Se o benefício for positivo em 6 meses, vale a pena automatizar.
```

```
---
```

## ## 2. CI/CD -- Pipelines de Integração e Deploy Contínuo

CI/CD é a espinha dorsal da automação em engenharia de software.

### ### Continuous Integration (CI)

Todo push para branches compartilhadas dispara:

1. Checkout do código
2. Instalação de dependências
3. Linting
4. Testes unitários
5. Testes de integração
6. Build
7. Análise de segurança

### ### Continuous Delivery / Continuous Deployment (CD)

- **\*\*Continuous Delivery:\*\*** O artefato é gerado e publicado em um ...
- **\*\*Continuous Deployment:\*\*** O deploy em produção é automático ap...

### ### GitHub Actions -- Exemplo Real

# **.github/workflows/ci.yml**

---

name: CI Pipeline

on:

push:

branches: [main, develop]

pull\_request:

branches: [main]

env:

NODE\_VERSION: 20.x

PNPM\_VERSION: 9

jobs:

lint:

name: Lint e Formatação

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v4

- uses: pnpm/action-setup@v3

with:

version: \${{ env.PNPM\_VERSION }}

- uses: actions/setup-node@v4

with:

node-version: \${{ env.NODE\_VERSION }}

cache: pnpm

- run: pnpm install --frozen-lockfile

- run: pnpm lint

test: run: pnpm format:check

name: Testes

needs: lint

runs-on: ubuntu-latest

```
services:
  postgres:
    image: postgres:16-alpine
    env:
      POSTGRES_DB: app_test
      POSTGRES_USER: app
      POSTGRES_PASSWORD: secret
    ports:
      - 5432:5432
    options: >-
      --health-cmd pg_isready
      --health-interval 10s
      --health-timeout 5s
      --health-retries 5
  steps:
    - uses: actions/checkout@v4
    - uses: pnpm/action-setup@v3
      with:
        version: ${{ env.PNPM_VERSION }}
    - uses: actions/setup-node@v4
      with:
        node-version: ${{ env.NODE_VERSION }}
        cache: pnpm
    - run: pnpm install --frozen-lockfile
    - run: pnpm test:unit
    - run: pnpm test:integration
    env:
      DATABASE_URL:
postgres://app:secret@localhost:5432/app_test
build:
```

```
name: Build
needs: test
runs-on: ubuntu-latest
steps:
  - uses: actions/checkout@v4
  - uses: pnpm/action-setup@v3
    with:
      version: ${{ env.PNPM_VERSION }}
  - uses: actions/setup-node@v4
    with:
      node-version: ${{ env.NODE_VERSION }}
      cache: pnpm
  - run: pnpm install --frozen-lockfile
  - run: pnpm build
  - uses: actions/upload-artifact@v4
    with:
      name: build-output
```

[text]

```
### GitLab CI -- Exemplo Real
```

# **.gitlab-ci.yml**

---

stages:

- lint
- test
- build

variables:

NODE\_VERSION: "20"

PNPM\_VERSION: "9"

cache:

key: \${CI\_COMMIT\_REF\_SLUG}

paths:

- node\_modules/

lint:pnpm-store/

stage: lint

image: node:\${NODE\_VERSION}

script:

- npm install -g pnpm@\${PNPM\_VERSION}
- pnpm install --frozen-lockfile
- pnpm lint

test:pnpm format:check

stage: test

image: node:\${NODE\_VERSION}

services:

- postgres:16-alpine

variables:

DATABASE\_URL: postgres://app:secret@postgres:5432/app\_test

POSTGRES\_DB: app\_test

```
  POSTGRES_USER: app
  POSTGRES_PASSWORD: secret
script:
  - npm install -g pnpm@${PNPM_VERSION}
  - pnpm install --frozen-lockfile
  - pnpm test:unit
  - pnpm test:integration
build:
stage: build
image: node:${NODE_VERSION}
script:
  - npm install -g pnpm@${PNPM_VERSION}
  - pnpm install --frozen-lockfile
  - pnpm build
artifacts:
  paths:
    - dist/
  expire_in: 30 days
deploy:
stage: deploy
image: alpine:latest
script:
  - apk add --no-cache curl
  - curl -X POST "${DEPLOY_HOOK_URL}"
only:
  - main
when: manual
environment: production
```

```
---
```

### ## 3. Automação de Testes

A pipeline de CI deve executar testes em camadas, respeitando a \*...

#### ### Testes Unitários

Executados primeiro -- são rápidos e isolados.

// Exemplo com vitest

```
import { describe, it, expect } from 'vitest'
import { calculateDiscount } from './pricing'
describe('calculateDiscount', () => {
  it('aplica 10% para compras acima de R$ 100', () => {
    expect(calculateDiscount(150)).toBe(135)
  })
  it('não aplica desconto para compras abaixo de R$ 100', () => {
    expect(calculateDiscount(50)).toBe(50)
  })
  it('lança erro para valores negativos', () => {
    expect(() => calculateDiscount(-10)).toThrow('Valor inválido')
  })
})
```

[text]

#### ### Testes de Integração

Testam a interação entre módulos, geralmente com banco de dados r...

```
import { describe, it, expect, beforeEach, afterEach } from 'vitest'
import { createApp } from './app'
import { prisma } from './lib/prisma'
const app = createApp()
describe('POST /users', () => {
```

```
beforeAll(async () => {
  await prisma.$executeRawUnsafe('TRUNCATE TABLE users
  CASCADE')
})
afterAll(async () => {
  await prisma.$disconnect()
})
it('cria um usuário com dados válidos', async () => {
  const response = await app.inject({
    method: 'POST',
    url: '/users',
    payload: { name: 'João', email: 'joao@email.com' },
  })
  expect(response.statusCode).toBe(201)
  expect(response.json()).toHaveProperty('id')
})
```

```
### Testes E2E
```

```
Testam o sistema como um todo, simulando o usuário real.
```

## job no CI para E2E

---

e2e:

```
name: Testes End-to-End
needs: build
runs-on: ubuntu-latest
steps:
  - uses: actions/checkout@v4
  - uses: pnpm/action-setup@v3
    with:
      version: ${{ env.PNPM_VERSION }}
  - uses: actions/setup-node@v4
    with:
      node-version: ${{ env.NODE_VERSION }}
      cache: pnpm
  - run: pnpm install --frozen-lockfile
  - run: pnpm build
  - run: pnpm dlx playwright install --with-deps
  - run: pnpm test:e2e
env:
```

```
// E2E com Playwright
```

```
import { test, expect } from '@playwright/test'
test('usuário consegue finalizar compra', async ({ page }) => {
  await page.goto('/produtos')
  await page.click('text=Adicionar ao carrinho')
  await page.click('text=Finalizar compra')
  await page.fill('[name=email]', 'joao@email.com')
```

```
await page.click('text=Confirmar');  
await expect(page.locator('text=Pedido confirmado')).toBeVisible()
```

```
---
```

```
## 4. Automação de Infraestrutura -- IaC
```

```
Infrastructure as Code (IaC) é o gerenciamento de infraestrutura ...
```

```
### Terraform (HashiCorp)
```

## main.tf

---

```
terraform {
  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
  backend "s3" {
    bucket = "meu-terraform-state"
    key    = "prod/terraform.tfstate"
    region = "us-east-1"
  }
  provider "aws" {
    region = "us-east-1"
  }
  resource "aws_ecs_service" "app" {
    name            = "app-service"
    cluster         = aws_ecs_cluster.main.id
    task_definition = aws_ecs_task_definition.app.arn
    desired_count   = 2
    launch_type     = "FARGATE"
    network_configuration {
      subnets         = var.private_subnet_ids
      security_groups = [aws_security_group.app.id]
    }
  }
  deployment_controller {
    type = "CODE_DEPLOY" # blue/green
  }
}
```

[text]

### Pulumi

IaC com linguagens de programação reais (TypeScript, Python, Go, ...

```
import * as aws from '@pulumi/aws'
import * as pulumi from '@pulumi/pulumi'
const config = new pulumi.Config()
const stack = pulumi.getStack()
const cluster = new aws.ecs.Cluster('app-cluster')
const taskDefinition = new aws.ecs.TaskDefinition('app-task', {
  family: 'app',
  cpu: '256',
  memory: '512',
  networkMode: 'awsvpc',
  executionRoleArn: config.require('executionRoleArn'),
  containerDefinitions: JSON.stringify([
    {
      name: 'app',
      image: 'meuregistro/app:${stack}',
      portMappings: [{ containerPort: 3000 }],
    },
  ]),
})
new aws.ecs.Service('app-service', {
  cluster: cluster.arn,
  taskDefinition: taskDefinition.arn,
  desiredCount: 2,
  launchType: 'FARGATE',
  networkConfiguration: {
    subnets: config.requireObject<string[]>('privateSubnets'),
    securityGroups: [config.require('securityGroupId')],
  },
})
```

`},`

```
### CloudFormation (AWS)
```

```
Template declarativo em YAML/JSON para recursos AWS.
```

# template.yaml

---

AWSTemplateFormatVersion: "2010-09-09"

Description: "Stack da aplicação"

Parameters:

Env:

Type: String

Default: production

Resources:

ECSCluster:

Type: AWS::ECS::Cluster

Properties:

ClusterName: !Sub "app-cluster-\${Env}"

AppService:

Type: AWS::ECS::Service

Properties:

ServiceName: !Sub "app-service-\${Env}"

Cluster: !Ref ECSCluster

LaunchType: FARGATE

DesiredCount: 2

TaskDefinition: !Ref AppTaskDefinition

NetworkConfiguration:

AwsvpcConfiguration:

Subnets:

- !Ref PrivateSubnet1

[text]

### Pipeline para IaC

iac-plan:

```
name: Terraform Plan
runs-on: ubuntu-latest
steps:
  - uses: actions/checkout@v4
  - uses: hashicorp/setup-terraform@v3
    with:
      terraform_version: 1.8.0
  - run: terraform init
  - run: terraform fmt -check
  - run: terraform validate
  - run: terraform plan -out=tfplan
  - uses: actions/upload-artifact@v4
    with:
      name: tfplan
      path: tfplan
iac-apply:
  name: Terraform Apply
  needs: iac-plan
  runs-on: ubuntu-latest
  if: github.ref == 'refs/heads/main'
  environment: production
  steps:
    - uses: actions/checkout@v4
    - uses: hashicorp/setup-terraform@v3
    - run: terraform init
    - uses: actions/download-artifact@v4
    with:
      name: tfplan
    - run: terraform apply tfplan
```

```
---
```

## ## 5. Automação de Deploys

### ### Blue-Green Deployment

Duas versões do ambiente (blue = atual, green = nova). O balancea...

USUÁRIOS -> Load Balancer -> Blue (v1.0) [OK]

Switch: DNS/ALB aponta para Green (v1.1) [green] (após validação)

### ### Canary Deployment

Uma porcentagem pequena do tráfego vai para a nova versão. Aos po...

# Apropriado para Kubernetes com Argo Rollouts

---

```
apiVersion: argoproj.io/v1alpha1
kind: Rollout
metadata:
  name: app-rollout
spec:
  replicas: 10
  revisionHistoryLimit: 2
  selector:
    matchLabels:
      app: meu-app
  template:
    metadata:
      labels:
        app: meu-app
    spec:
      containers:
        - name: app
          image: meuregistro/app:1.1.0
  strategy:
    canary:
      steps:
        - setWeight: 10
        - pause: { duration: 5m }
        - setWeight: 50
```

- pause: { duration: 5m }

[text]

### Rolling Update

Substitui instâncias gradualmente, sem tempo de inatividade.

# docker-compose.yml com rolling update no Swarm

---

```
version: "3.8"
services:
  app:
    image: meuregistro/app:${VERSION}
    deploy:
      replicas: 3
      update_config:
        parallelism: 1
        delay: 10s
        order: start-first
      rollback_config:
        parallelism: 1
```

```
### Feature Flags
```

```
Permite ativar/desativar funcionalidades em produção sem fazer d...
```

```
import { createClient } from '@config/sdk'
const config = createClient({ apiKey: process.env.CONFIGU_KEY })

async function handler(req, res) {
  const novoCheckout = await config.getFlag('checkout-v2-enabled')
  if (novoCheckout && req.user.id in novoCheckout.targeting) {
    return handleCheckoutV2(req, res)
  }
  return handleCheckoutV1(req, res)
}
```

[text]

**\*\*Ferramentas:\*\*** LaunchDarkly, Configu, GrowthBook, Unleash, Flag...

---

**##** 6. Automação de Banco de Dados

**###** Migrations Automáticas no CI

# Job de migration

---

migrate:

name: Rodar Migrations

needs: build

runs-on: ubuntu-latest

environment: \${{ github.ref == 'refs/heads/main' && 'production' || 'staging' }}

steps:

- uses: actions/checkout@v4
- uses: pnpm/action-setup@v3
- uses: actions/setup-node@v4
- run: pnpm install --frozen-lockfile
- run: pnpm build
- uses: actions/download-artifact@v4

with:

name: build-output

path: dist

- run: pnpm db:migrate

env:

```
### Exemplo com Prisma Migrate
```

```
// scripts/migrate.ts
```

```
import { execSync } from 'node:child_process'
const env = process.env.NODE_ENV || 'development'
```

```
function run() {
  // console.log(> Rodando migrations em ${env})
  // Validar se a migration é segura (não dropa colunas sem verificação)
```

```
execSync('pnpm prisma migrate deploy', {
```

```
    stdio: 'inherit',
    env: { ...process.env, DATABASE_URL:
process.env.DATABASE_URL },
  })
  console.log('[OK] Migrations executadas com sucesso')
}
```

[text]

### Seed Automático em CI

```
// scripts/seed.ts
import { PrismaClient } from '@prisma/client'
const prisma = new PrismaClient()
async function seed() {
  console.log('[seedling] Iniciando seed...')
  await prisma.tenant.create({
    data: {
      name: 'Default',
      slug: 'default',
      users: {
        create: {
          email: 'admin@empresa.com',
          role: 'ADMIN',
        },
      },
    },
  })
  console.log('[OK] Seed concluído')
}
seed()
  .catch((e) => {
    console.error(e)
    process.exit(1)
  })
})
```

```
### Rollback Automático
```

```
Estratégia: antes de rodar a migration, fazer backup; se a migrat...
```

# Rollback automático

---

- name: Backup antes da migration

```
run: pg_dump $DATABASE_URL > /tmp/pre_migration_backup.sql
```

```
env:
```

```
  DATABASE_URL: ${ secrets.DATABASE_URL }
```

- name: Rodar migration

```
id: migrate
```

```
run: pnpm db:migrate
```

```
continue-on-error: true
```

```
env:
```

```
  DATABASE_URL: ${ secrets.DATABASE_URL }
```

- name: Rollback em caso de falha

```
if: steps.migrate.outcome != 'success'
```

```
run: |
```

```
  echo "[refresh] Migration falhou -- restaurando backup..."
```

```
  psql $DATABASE_URL < /tmp/pre_migration_backup.sql
```

```
  echo "[OK] Backup restaurado"
```

```
env:
```

```
  DATABASE_URL: ${ secrets.DATABASE_URL }
```

[text]

---

## ## 7. Automação de Segurança

### ### SAST (Static Application Security Testing)

Análise estática de segurança diretamente no pipeline.

#### security-sast:

name: Análise Estática (SAST)

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v4

- uses: pnpm/action-setup@v3

- uses: actions/setup-node@v4

~~run: pnpm install --frozen-lockfile~~  
# ESLint com plugins de segurança

- name: ESLint Security

~~run: pnpm dlx eslint . --ext .ts --rulesdir eslint-security-rules~~  
# SonarQube

- name: SonarQube Scan

uses: sonarsource/sonarcloud-github-action@master

env:

SONAR\_TOKEN: \${ secrets.SONAR\_TOKEN }

with:

args: >

- Dsonar.organization=minha-org

- Dsonar.projectKey=meu-projeto

~~-Dsonar.javascript.lcov.reportPaths=coverage/lcov.info~~  
# Trivy para vulnerabilidades em container

- name: Trivy Scan

```
uses: aquasecurity/trivy-action@master
with:
  image-ref: meuregistro/app:${{ github.sha }}
  format: sarif
```

```
### DAST (Dynamic Application Security Testing)
```

```
Testa a aplicação rodando, simulando ataques reais.
```

security-dast:

```
name: Teste Dinâmico (DAST)
needs: deploy-staging
runs-on: ubuntu-latest
steps:
  - name: OWASP ZAP Scan
    uses: zaproxy/action-full-scan@v0.10
    with:
      target: https://staging.meusistema.com.br
      rules_file_name: .zap/rules.tsv
```

```
[text]
```

```
### Dependency Scanning
```

dependency-scan:

```
name: Varredura de Dependências
runs-on: ubuntu-latest
steps:
  - uses: actions/checkout@v4
  - uses: pnpm/action-setup@v3
```

```
- uses: actions/setup-node@v4
run: pnpm install --frozen-lockfile
# Snyk
- name: Snyk Scan
  uses: snyk/actions/node@master
  env:
    SNYK_TOKEN: ${ secrets.SNYK_TOKEN }
  with:
    args: --severity-threshold=medium
# Dependabot alerts (nativo do Github)
# npm audit
- name: npm audit
```

```
---

## 8. Automação de Code Review

### Bots no Pipeline
```

```
code-review:
  name: Revisão Automática
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
      with:
        fetch-depth: 0
# Linters automáticos
- uses: pnpm/action-setup@v3
- uses: actions/setup-node@v4
- run: pnpm install --frozen-lockfile
- run: pnpm lint
run: pnpm format:check
# Status checks
```

```
- name: Verificar tamanho do PR
uses: actions/github-script@v7
with:
  script: |
    const { data: pr } = await github.rest.pulls.get({
      ...context.repo,
      pull_number: context.issue.number,
    })
    const changedFiles = pr.changed_files
    const additions = pr.additions
    if (changedFiles > 20) {
      core.warning([warning] PR grande: ${changedFiles} arquivos
alterados)
    }
    if (additions > 500) {
      core.warning([warning] Muitas linhas adicionadas:
${additions})
    }
# CodeQL
- name: Initialize CodeQL
  uses: github/codeql-action/init@v3
  with:
    languages: javascript-typescript
- name: CodeQL Analysis
  uses: github/codeql-action/analyze@v3
  with:
    category: /language:javascript
[text]

### Dependabot Configuration
```

# **.github/dependabot.yml**

---

version: 2

updates:

- package-ecosystem: "npm"

  - directory: "/"

- schedule:

  - interval: "weekly"

  - day: "monday"

- open-pull-requests-limit: 10

labels:

- "dependencies"

- "automated"

reviewers:

- "time-squad"

commit-message:

- prefix: "chore"

```
---
```

```
## 9. Automação de Releases
```

```
### Semantic Versioning Automático
```

release:

- name: Gerar Release

- needs: build

- runs-on: ubuntu-latest

- if: github.ref == 'refs/heads/main'

```
permissions:
  contents: write
  packages: write
steps:
  - uses: actions/checkout@v4
    with:
      fetch-depth: 0
  # Analise de commits para versionamento semântico
  - name: Semantic Release
    uses: cycjimmy/semantic-release-action@v4
    id: semantic
    env:
      GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }
      NPM_TOKEN: ${ secrets.NPM_TOKEN }
    with:
      extra_plugins: |
        @semantic-release/changelog
        @semantic-release/git
        @semantic-release/npm
  - name: Publicar no npm
    if: steps.semantic.outputs.new_release_published == 'true'
    run: |
      echo "[package] Publicando versão ${
steps.semantic.outputs.new_release_version }"
      npm publish
    env:
      NODE_AUTH_TOKEN: ${ secrets.NPM_TOKEN }
```

```
[text]
```

```
### Conventional Commits
```

```
O `semantic-release` depende do padrão **Conventional Commits**:
```

feat: adiciona calculadora de impostos

feat(api): nova rota de relatórios

fix: corrige validação de CPF

fix(auth): timeout na renovação do token

chore: atualiza dependências

docs: atualiza README

refactor: extrai lógica de pagamento para serviço

perf: otimiza consulta de histórico

```
### Changelog Gerado Automaticamente
```

```
O `semantic-release` com plugin `@semantic-release/changelog` ger...
```

# Changelog

---

## [1.5.0](https://github.com/org/repo/compare/v1.4.0...v1.5.0) (2025-06-15)

### Features

1. **\*\*pagamento:\*\*** adiciona suporte a PIX ([abc1234](https://github.com/org/repo/commit/abc1234))
2. **\*\*relatorios:\*\*** nova rota de exportação CSV ([def5678](https://github.com/org/repo/commit/def5678))

### Bug Fixes

1. **\*\*auth:\*\*** corrige timeout na renovação do token ([ghi9012](https://github.com/org/repo/commit/ghi9012))
2. **\*\*validacao:\*\*** CPF com formatação agora é aceito ([jkl3456](https://github.com/org/repo/commit/jkl3456))

---

## 10. Automação de Ambientes

### Ephemeral Environments

Ambientes temporários criados automaticamente para cada branch de...

deploy-preview:

```
name: Deploy Preview
runs-on: ubuntu-latest
if: github.event_name == 'pull_request'
environment:
  name: preview-${{ github.event.number }}
  url: https://pr-${{ github.event.number }}.meusistema.com.br
steps:
  - uses: actions/checkout@v4
  - uses: pnpm/action-setup@v3
  - uses: actions/setup-node@v4
  - run: pnpm install --frozen-lockfile
  - run: pnpm build
# Criar banco de dados
  - name: Criar database preview
    run: |
      aws rds create-db-instance \
        --db-instance-identifier app-preview-${{ github.event.number }} \
        --db-instance-class db.t3.micro \
        --engine postgres \
        --master-username preview \
        --master-user-password ${secrets.PREVIEW_DB_PASSWORD}
  - name: Rodar migrations
    run: pnpm db:migrate
  env:
    DATABASE_URL: postgres://preview:${secrets.PREVIEW_DB_PASSWORD}
    @preview_db/app-preview-${{ github.event.number }}
  - name: Fazer deploy no ECS
    run: |
```

```
aws ecs update-service \
  --cluster preview \
  --service app-preview-${{ github.event.number }} \
  --force-new-deployment
destroy-preview:
  name: Destruir Preview
  runs-on: ubuntu-latest
  if: github.event_name == 'pull_request' && github.event.action ==
'closed'
  steps:
    - name: Remover database preview
      run: |
        aws rds delete-db-instance \
          --db-instance-identifier app-preview-${{ github.event.number
}} \
          --skip-final-snapshot
    - name: Remover serviço ECS
      run: |
        aws ecs delete-service \
          --cluster preview \
          --service app-preview-${{ github.event.number }} \
```

[text]

```
### Preview Deployments (Vercel / Render)
```

```
vercel-preview:
  name: Vercel Preview
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: amondnet/vercel-action@v25
```

with:

```
vercel-token: ${ secrets.VERCEL_TOKEN }
vercel-org-id: ${ secrets.VERCEL_ORG_ID }
vercel-project-id: ${ secrets.VERCEL_PROJECT_ID }
github-token: ${ secrets.GITHUB_TOKEN }
```

```
---
```

```
## 11. Automação de Monitoramento
```

```
### Alertas Automáticos
```

monitoring-setup:

name: Configurar Alertas

runs-on: ubuntu-latest

steps:

- name: Configurar Datadog Monitors

uses: datadog/synthetics-ci-github-action@v1

with:

api\_key: \${ secrets.DD\_API\_KEY }

app\_key: \${ secrets.DD\_APP\_KEY }

- name: Configurar alerta de latência P99  
public\_ids: \${ vars.DATADOG\_SYNTHETIC\_TESTS }

run: |

```
curl -X POST "https://api.datadoghq.com/api/v1/monitor" \
-H "Content-Type: application/json" \
-H "DD-API-KEY: ${ secrets.DD_API_KEY }" \
-H "DD-APPLICATION-KEY: ${ secrets.DD_APP_KEY }" \
-d '{
  "name": "Latência P99 > 500ms",
  "type": "query alert",
```

```

    "query":
      "avg(last_5m):p99(trace.http.request.duration{service:app}) > 500",
      "message": "Latência alta detectada! @time-squad",
      "tags": ["service:app", "severity:critical"]

```

[text]

### Self-Healing

```

// scripts/self-healing.ts
import { ECSClient, DescribeServicesCommand,
UpdateServiceCommand } from '@aws-sdk/client-ecs'
const ecs = new ECSClient({ region: 'us-east-1' })
async function healService(cluster: string, service: string) {
  const { services } = await ecs.send(
    new DescribeServicesCommand({ cluster, services: [service] })
  )
  const svc = services![0]
  const running = svc.runningCount ?? 0
  const desired = svc.desiredCount ?? 0
  if (running < desired) {
    console.log([wrench] Service ${service} com ${running}/${desired}
forçando novo deploy...)
    await ecs.send(
      new UpdateServiceCommand({
        cluster,
        service,
        forceNewDeployment: true,
      })
    )
    console.log('[OK] Deploy forçado enviado')
  } else {
    console.log([OK] Service ${service} saudavel
(${running}/${desired}))
  }
}

```

1

```
### Runbooks Automáticos
```

# runbook: recuperacao-banco.yml

---

name: Runbook - Recuperar Database

on:

workflow\_dispatch:

inputs:

database\_name:

description: 'Nome do banco'

required: true

restore\_point:

description: 'PITR timestamp (YYYY-MM-DD HH:MM:SS)'

jobs:required: false

restore:

runs-on: ubuntu-latest

steps:

- name: Validar parâmetros

run: |

if [ -z "\${{ inputs.database\_name }}" ]; then

echo "[X] database\_name é obrigatório"

exit 1

- name: Parar aplicação

run: |

aws ecs update-service \

--cluster production \

--service app-service \

- name: Restaurar snapshot

run: |

```
if [ -n "${inputs.restore_point}" ]; then
  aws rds restore-db-instance-to-point-in-time \
    --source-db-instance-identifier "${inputs.database_name}" \
    --target-db-instance-identifier "${inputs.database_name}
}}-restored \
    --restore-time "${inputs.restore_point}"
else
  SNAPSHOT=$(aws rds describe-db-snapshots \
    --db-instance-identifier "${inputs.database_name}" \
    --query "DBSnapshots[-1].DBSnapshotIdentifier" \
    --output text)
  aws rds restore-db-instance-from-db-snapshot \
    --db-instance-identifier "${inputs.database_name}-restored
\
    --db-snapshot-identifier $SNAPSHOT
fi
- name: Trocar DNS para DB restaurado
  run: |
    aws rds modify-db-instance \
      --db-instance-identifier "${inputs.database_name}" \
      --new-db-instance-identifier "${inputs.database_name}-old
    aws rds modify-db-instance \
      --db-instance-identifier "${inputs.database_name}-restored
\
      --new-db-instance-identifier "${inputs.database_name}"
- name: Subir aplicação
  run: |
    aws ecs update-service \
      --cluster production \
      --service app-service \
```

```
- name: Notificar  
  --desired-count 3
```

```
  run: |
```

```
    curl -X POST ${{ secrets.SLACK_WEBHOOK }} \
```

```
    -H "Content-Type: application/json" \
```

```
    -d '{"text": "[OK] Database ${{ inputs.database_name }}"}
```

```
[text]
```

```
---
```

```
## 12. Pipeline as Code -- Boas Práticas
```

```
### Modularização e Reuso
```

# **.github/actions/setup-node/action.yml**

---

name: "Setup Node.js"

description: "Configura Node.js, pnpm e cache"

inputs:

node-version:

required: false

default: "20.x"

pnpm-version:

required: false

runs: default: "9"

using: "composite"

steps:

- uses: actions/checkout@v4

- uses: pnpm/action-setup@v3

with:

version: \${{ inputs.pnpm-version }}

- uses: actions/setup-node@v4

with:

node-version: \${{ inputs.node-version }}

cache: pnpm

- run: pnpm install --frozen-lockfile

shell: bash

*.github/actions/setup-node/action.yml | IA para Desenvolvedores*

Uso da action reutilizável:

## **.github/workflows/ci.yml**

---

```
jobs:
  lint:
    runs-on: ubuntu-latest
    steps:
      - uses: ./github/actions/setup-node
  test:
    run: pnpm lint
    runs-on: ubuntu-latest
    steps:
      - uses: ./github/actions/setup-node
```

[text]

```
### Reuso de Jobs com Matrix
```

```
test-matrix:
  name: Testes (${{ matrix.node-version }})
  runs-on: ubuntu-latest
  strategy:
    matrix:
      node-version: [18.x, 20.x, 22.x]
    fail-fast: false
  steps:
    - uses: ./github/actions/setup-node
      with:
        node-version: ${{ matrix.node-version }}
    - run: pnpm test:ci
```

### ### Pipeline de Deploy com Environments e Gates

deploy-production:

name: Deploy em Produção

needs: [lint, test, build, security-sast]

runs-on: ubuntu-latest

if: github.ref == 'refs/heads/main'

environment:

name: production

url: https://meusistema.com.br

concurrency:

group: production-deploy

cancel-in-progress: false

steps:

- uses: actions/checkout@v4

- uses: ./github/actions/setup-node

- uses: actions/download-artifact@v4

with:

name: build-output

path: dist

- name: Login no Docker Registry

uses: docker/login-action@v3

with:

username: \${{ secrets.DOCKER\_USER }}

password: \${{ secrets.DOCKER\_PASS }}

- name: Build e Push da Imagem

run: |

docker build -t meuregistro/app:\${{ github.sha }} .

docker push meuregistro/app:\${{ github.sha }}

- name: Deploy no ECS

```
run: |
  aws ecs update-service \
    --cluster production \
    --service app-service \
    --force-new-deployment
- name: Verificar Saúde
  run: |
    for i in {1..30}; do
      STATUS=$(curl -s -o /dev/null -w "%{http_code}"
https://meusistema.com.br/health)
      if [ "$STATUS" = "200" ]; then
        echo "[OK] Deploy saudável"
        exit 0
      fi
      sleep 10
    done
    echo "[X] Health check falhou após deploy"
    exit 1
- name: Rollback automático
  if: failure()
  run: |
    echo "[refresh] Iniciando rollback..."
    aws ecs update-service \
      --cluster production \
      --service app-service \
      --force-new-deployment
```